

Multiplicação de Matriz por Vetor: Localidade Temporal e Espacial no Acesso à Memória

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório apresenta a implementação de um programa em linguagem C para comparar duas formas de percorrer os elementos de uma matriz durante a multiplicação de matriz por vetor (MxV), avaliando o impacto do padrão de acesso à memória sobre o desempenho:

- **Versão por linhas:** o laço externo percorre as linhas i e o laço interno percorre as colunas j , de modo que os elementos $A[i][j]$ são acessados sequencialmente na memória (*stride* 1).
- **Versão por colunas:** o laço externo percorre as colunas j e o laço interno percorre as linhas i , de modo que os elementos $A[i][j]$ são acessados com um salto de N posições a cada passo (*stride* N), pois em C as matrizes são armazenadas em ordem *row-major* (por linhas).

O objetivo é identificar a partir de que tamanho de matriz os tempos de execução das duas versões passam a divergir de forma significativa e explicar essa divergência com base no funcionamento da memória *cache*.

2 Metodologia

O programa aloca uma matriz quadrada $N \times N$ como um único bloco contíguo de memória e mede o tempo de execução de cada versão do algoritmo com a função `clock_gettime()` (relógio `CLOCK_MONOTONIC` da biblioteca `time.h`), que oferece alta resolução e não é afetada por ajustes do relógio do sistema. Para cada tamanho de matriz N (variando entre 64 e 16384, em potências de 2), cada versão foi executada 5 vezes, sendo registrado o menor tempo obtido, de forma a reduzir o ruído de medição causado por outras tarefas do sistema operacional. O código completo, compilado com `gcc -O2 -o tarefa2 tarefa2.c`, é apresentado no Código 1.

3 Resultados

A Tabela 1 apresenta os tempos de execução (em segundos, melhor caso entre 5 repetições) das duas versões do algoritmo, além da razão entre o tempo da versão por colunas e o tempo da versão por linhas.

Tabela 1: Tempos de execução da multiplicação $M \times V$ por linhas e por colunas

N	Linhas (s)	Colunas (s)	Razão (col./lin.)
64	0,000004	0,000004	1,03×
128	0,000018	0,000017	0,94×
256	0,000075	0,000144	1,91×
512	0,000222	0,001498	6,75×
1.024	0,000938	0,008236	8,78×
2.048	0,003564	0,051392	14,42×
4.096	0,014300	0,379421	26,53×
8.192	0,057248	3,403755	59,46×
16.384	0,267308	8,399817	31,42×

4 Análise

4.1 Ponto de divergência dos tempos

Para matrizes pequenas ($N = 64$ e $N = 128$), os tempos das duas versões são praticamente idênticos (razão próxima de $1\times$). A partir de $N = 256$ a diferença começa a aparecer (razão de aproximadamente $1,9\times$) e cresce de forma acentuada e consistente até $N = 8192$, onde a versão por colunas chega a ser quase 60 vezes mais lenta que a versão por linhas. Pode-se dizer, portanto, que a divergência *significativa* de desempenho passa a ocorrer a partir de $N \approx 256$ –512, tornando-se dramática para $N \geq 2048$.

4.2 Relação com a hierarquia de memória (cache)

O comportamento observado é explicado diretamente pelo conceito de **localidade espacial** e pela forma como a matriz é armazenada na memória. Em C, uma matriz bidimensional é organizada em ordem *row-major*: os elementos de uma mesma linha ficam contíguos na memória, enquanto elementos de uma mesma coluna estão separados por N posições (uma linha inteira).

Na versão por linhas, o acesso sequencial (*stride* 1) faz com que, ao carregar um elemento, todo o bloco de memória ao seu redor (a *cache line*, tipicamente 64 bytes, ou seja, 8 valores `double`) seja aproveitado nos acessos seguintes, gerando muitos *cache hits*. Já na versão por colunas, cada acesso salta N posições (*stride* N): para valores de N suficientemente grandes, cada acesso cai em uma linha de cache diferente, gerando *cache misses* frequentes e obrigando o processador a buscar dados repetidamente na memória principal, muito mais lenta que a cache.

4.3 Por que a diferença só aparece a partir de certo tamanho

Para matrizes pequenas, o conjunto de dados inteiro cabe (ou quase cabe) nas caches L1/L2 do processador, de modo que mesmo os acessos com *stride* grande acabam sendo atendidos pela cache na maior parte do tempo. À medida que N cresce, o tamanho da matriz ($N^2 \times 8$ bytes) ultrapassa a capacidade das caches L1, depois L2 e eventualmente L3, e a taxa de *cache misses* da versão por colunas aumenta continuamente – exatamente o efeito observado no crescimento da razão entre os tempos até $N = 8192$. A leve queda da razão em $N = 16384$ ocorre porque, nesse ponto, mesmo a versão por linhas já não

cabe inteiramente nas caches mais rápidas, de modo que seu tempo também passa a ser dominado por acessos à memória principal, reduzindo a vantagem relativa entre as duas versões.

4.4 Localidade temporal

Além da localidade espacial, há também um componente de localidade **temporal** relevante: o vetor v é reutilizado em todas as iterações do laço. Na versão por linhas, cada elemento $v[j]$ é acessado repetidamente em iterações próximas, favorecendo sua permanência na cache, e a variável acumuladora **soma** é mantida em registrador durante toda a soma de uma linha. Esses fatores reforçam ainda mais a vantagem da versão por linhas sobre a versão por colunas.

5 Reflexão: aplicações reais

O padrão observado neste experimento – programas com a mesma complexidade assintótica apresentando desempenho drasticamente diferente conforme o padrão de acesso à memória – se repete em diversas áreas da computação:

- **Álgebra linear e computação científica:** bibliotecas de alto desempenho (como BLAS/LAPACK) reorganizam laços e utilizam técnicas de *blocking/tiling* exatamente para maximizar a reutilização de dados na cache em operações com matrizes grandes, obtendo ganhos de desempenho semelhantes aos observados aqui.
- **Bancos de dados:** o formato de armazenamento de tabelas (orientado a linhas ou a colunas) é escolhido de acordo com o padrão de acesso predominante da aplicação – sistemas OLTP tendem a usar armazenamento por linhas, enquanto sistemas analíticos (OLAP) costumam usar armazenamento por colunas, cada um otimizado para seu próprio padrão de varredura.
- **Processamento de imagens e aprendizado de máquina:** operações sobre tensores multidimensionais (convoluções, multiplicações de matrizes em redes neurais) são altamente sensíveis à ordem de acesso à memória; por isso, frameworks como cuDNN e cuBLAS investem pesadamente em otimizações de localidade de memória

A Código-Fonte

Código utilizado para gerar os resultados apresentados neste relatório.

```
1  /*
2  * mxv.c - Multiplicação Matriz x Vetor (MxV): comparação de padrões de acesso
3  *
4  * Versão 1 (por linhas): laço externo percorre linhas i,
5  *                      laço interno percorre colunas j
6  *                      -> acessa A[i][j] sequencialmente (stride 1)
7  *
8  * Versão 2 (por colunas): laço externo percorre colunas j,
9  *                      laço interno percorre linhas i
10 *                      -> acessa A[i][j] pulando N elementos a cada
11 *                      passo (stride N), pois em C a matriz é
12 *                      armazenada em ordem row-major.
13 *
14 * Compilar: gcc -O2 -o mxv mxv.c
15 * Executar: ./mxv
16 */
17
18 #include <stdio.h>
19 #include <stdlib.h>
20 #include <time.h>
21
22 /* Aloca matriz N x N como um único bloco contíguo (row-major) */
23 static double **aloca_matriz(int n) {
24     double **A = malloc(n * sizeof(double *));
25     double *dados = malloc((size_t)n * n * sizeof(double));
26     for (int i = 0; i < n; i++)
27         A[i] = &dados[(size_t)i * n];
28     return A;
29 }
30
31 static void libera_matriz(double **A) {
32     free(A[0]);
33     free(A);
34 }
35
36 static void preenche(double **A, double *v, int n) {
37     for (int i = 0; i < n; i++) {
38         v[i] = (double)(rand() % 100) / 7.0;
39         for (int j = 0; j < n; j++)
40             A[i][j] = (double)(rand() % 100) / 3.0;
41     }
42 }
43
44 /* Versão por LINHAS: laço interno varia a coluna j (acesso sequencial) */
45 static void mxv_por_linhas(double **A, double *v, double *r, int n) {
46     for (int i = 0; i < n; i++) {
47         double soma = 0.0;
48         for (int j = 0; j < n; j++)
49             soma += A[i][j] * v[j];
50         r[i] = soma;
51     }
52 }
53
54 /* Versão por COLUNAS: laço interno varia a linha i (acesso com stride n) */
```

```

55 static void mxv_por_colunas(double **A, double *v, double *r, int n) {
56     for (int i = 0; i < n; i++)
57         r[i] = 0.0;
58     for (int j = 0; j < n; j++)
59         for (int i = 0; i < n; i++)
60             r[i] += A[i][j] * v[j];
61 }
62
63 /* Retorna tempo em segundos, alta resolução */
64 static double agora(void) {
65     struct timespec ts;
66     clock_gettime(CLOCK_MONOTONIC, &ts);
67     return ts.tv_sec + ts.tv_nsec * 1e-9;
68 }
69
70 int main(int argc, char **argv) {
71     int tamanhos_padrao[] = {64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384};
72     int n_tam = sizeof(tamanhos_padrao) / sizeof(tamanhos_padrao[0]);
73     int repeticoes = 5;
74
75     printf("%10s %14s %14s %10s\n",
76           "N", "linhas (s)", "colunas (s)", "razao");
77     printf("-----\n");
78
79     for (int t = 0; t < n_tam; t++) {
80         int n = tamanhos_padrao[t];
81         double **A = aloca_matriz(n);
82         double *v = malloc(n * sizeof(double));
83         double *r = malloc(n * sizeof(double));
84
85         srand(42);
86         preenche(A, v, n);
87
88         double melhor_linhas = 1e18, melhor_colunas = 1e18;
89
90         for (int rep = 0; rep < repeticoes; rep++) {
91             double t0 = agora();
92             mxv_por_linhas(A, v, r, n);
93             double t1 = agora();
94             if (t1 - t0 < melhor_linhas) melhor_linhas = t1 - t0;
95         }
96
97         for (int rep = 0; rep < repeticoes; rep++) {
98             double t0 = agora();
99             mxv_por_colunas(A, v, r, n);
100             double t1 = agora();
101             if (t1 - t0 < melhor_colunas) melhor_colunas = t1 - t0;
102         }
103
104         printf("%10d %14.6f %14.6f %10.2fx\n",
105               n, melhor_linhas, melhor_colunas,
106               melhor_colunas / melhor_linhas);
107
108         libera_matriz(A);
109         free(v);
110         free(r);
111     }
112

```

```
113     return 0;
114 }
```

Código 1: Código-fonte `tarefa2.c` – multiplicação matriz-vetor com dois padrões de acesso à memória.